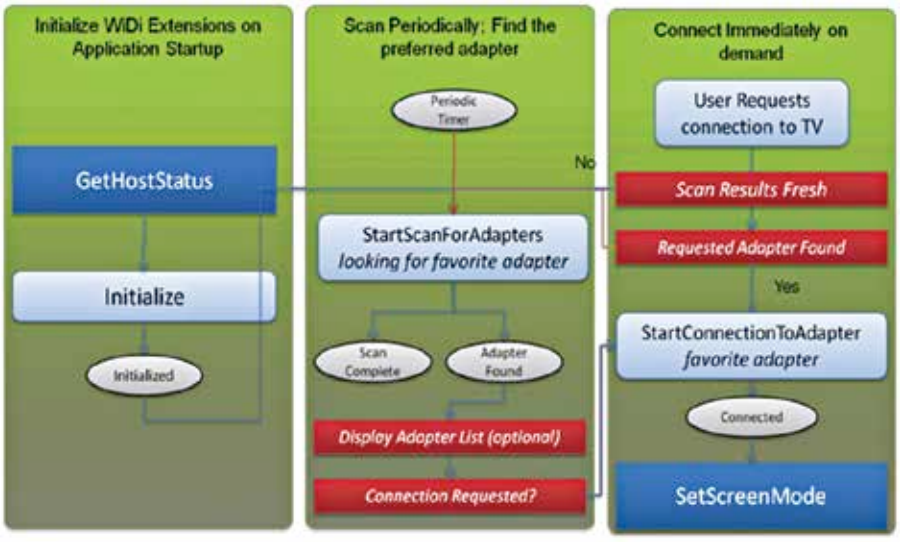

An optimized connection flow for a WiDi application

http://downloadcenter.intel.com/Detail_Desc.aspx?lang=eng&DwnldID=20410.

2 Unzip the archive to a temporary location on the target platform. Two files are required: Intel WiDi Extensions Library Setup.msi and setup.exe. Run the setup.exe program and follow the instructions in the installer. This will place all necessary files onto your system into `c:\Program Files(x86)\Intel Corporation\ Intel WiDi Extensions SDK`

3 Navigate to `C:\Program Files (x86)\Intel Corporation\Intel WiDi Extensions SDK\Intel WiDi Extensions Test Application`

4 Double-click VerifyWiDiExtensions.exe to start the test application.

LIFECYCLE OF A WIDI APP

Since demonstrating a sample program or application can get very lengthy, let's go through the overall lifecycle or flow through which a WiDi application handles the connection to the several WiDi adaptors or devices. This often involves scanning for multiple devices and connecting to a specific device which has been found.

GET HOST CAPABILITY

In short, this method queries for the capabilities of the host and determines if WiDi is supported or not. If yes, then it can also query for the version of the WiDi that is supported. A sample call to this method in c# can be made as follows: -

```
WiDi.GetHostCapability(bsKey, out bsValue);
```

Here bsKey is the value of the capability being queried. It can either be WiDiSupported or WiDiVersion. Each of these is self explanatory and return a value of Yes/No or the version of WiDi respectively.

GET HOST STATUS

This method returns the host's connection status information, i.e. whether the Agent is loaded or not, whether the connection is in use and which WiDiAdaptor it is connected to. The bsKey can be passed as the input parameter to the method, which returns the appropriate values for the connection information. A sample call to the method can be made as follows: -

```
WiDi.GetHostStatus(bsKey, out bsValue);
```

INITIALIZE

This method initializes the main application that is used to handle all the operations related to the connectivity mechanism.

A handle for messages to be received as callbacks can be passed as an input parameter. Any notification for expected or unexpected events is passed with the help of these callback methods. Callbacks supported are Initialized, InitializationFailed, AdaptorDiscovered, ScanComplete, ConnectionFailed, Connected, DisconnectFailed and Diconnected.

A block of code that tries to initialize the WiDi application looks as follows: -

```
try
                {
                    buttonStateDisabled();
                WiDi.Initialize((uint)Handle.ToInt64());
                    lstDebug.Items.Add("Initialization: S_SUCCESS \n");
                }
            catch (COMException ex)
                {
                    string sTemp;
                convertWiDiResulttoString(ex.ErrorCode, out sTemp);
                    lstDebug.Items.Add("Initialization Failed, error code  " + sTemp + "\n");
                    buttonStateUninitialized();
                }
                catch (Exception ex)
                {
                    MessageBox.Show("An unknown error occurred during startup. Please contact support." + ex.ToString());
                    CancelForm = true;
                }
```

Here the wParam of the message indicates the reason of the failure of initialization with the help of the convertWiDiResulttoString method which parses the enum for the status of the message to the string.

START SCAN FOR ADAPTORS

Upon success of initialization, the application can choose to start scanning for adaptors. This will result in callbacks in the form of AdaptorDiscovered or ScanComplete. A call to this method does not require any input or output parameters. It can be done as follows: -

```
WiDi.StartScanForAdaptors();
```

The success or failure of this scan can be handled in the form similar to the try/catch blocks as mentioned before.

GET ADAPTOR LIST

Once an adaptor is found after the scan, the GetAdaptorList method can be used to query the list of available adaptors. It allows restricting the list according to the connection status and the type of adaptor (audio/video/all). It has two input parameters and one output parameter which is a list of comma-separated values for the adaptors found: -

```
WiDi.GetAdapterList(filterName, typeName, out adapterList);
```

Here the filtername can be either Connected or AllOnTheNetwork. The typename refers to the list of audio, video or all adaptors required.